Esercizi d’esame su Stream API

Esercizio - Movie

- Dato il seguente codice:

```
import java.util.*;
import java.util.stream.*;
public class Main {
   public static void main(String[] args) {
      List<Movie> movies = Arrays.asList(
         new Movie("Inception", "Christopher Nolan", "Sci-Fi", 2010, 148),
         new Movie("The Godfather", "Francis Ford Coppola", "Crime", 1972, 175),
         new Movie("Interstellar", "Christopher Nolan", "Sci-Fi", 2014, 169),
         new Movie("Pulp Fiction", "Quentin Tarantino", "Crime", 1994, 154),
         new Movie("The Dark Knight", "Christopher Nolan", "Action", 2008, 152),
         new Movie("Django Unchained", "Quentin Tarantino", "Western", 2012, 165),
         new Movie("Tenet", "Christopher Nolan", "Sci-Fi", 2020, 150)
      );
      // TODO: Calcola e stampa la durata media dei film per regista
   }
}
```

Esercizio - Movie

```
class Movie {
    private String title;
    private String director;
    private String genre;
    private int year;
    private int duration; // in minuti
    public Movie(String title, String director, String genre, int year, int duration) {
        this.title = title;
        this.director = director;
        this.genre = genre;
        this.year = year;
        this.duration = duration;
    }
    public String getTitle() { return title; }
    public String getDirector() { return director; }
    public String getGenre() { return genre; }
    public int getYear() { return year; }
    public int getDuration() { return duration; }
    @Override
    public String toString() {
        return String.format(
            "%s (%d) - %s - %s - %d min",
            title, year, director, genre, duration
        );
    }
}
```

Esercizio - Movie

- Utilizzando Stream API, calcola e stampa:
- la durata media dei film per ciascun regista
- Output atteso (ordine non importante):

Christopher Nolan -> 154.75 minuti

Francis Ford Coppola -> 175.0 minuti

Quentin Tarantino -> 159.5 minuti

Esercizio - Movie - Soluzione

```
import java.util.*;
import java.util.stream.*;
public class Main {
   public static void main(String[] args) {
      List<Movie> movies = Arrays.asList(
         new Movie("Inception", "Christopher Nolan", "Sci-Fi", 2010, 148),
         new Movie("The Godfather", "Francis Ford Coppola", "Crime", 1972, 175),
         new Movie("Interstellar", "Christopher Nolan", "Sci-Fi", 2014, 169),
         new Movie("Pulp Fiction", "Quentin Tarantino", "Crime", 1994, 154),
         new Movie("The Dark Knight", "Christopher Nolan", "Action", 2008, 152),
         new Movie("Django Unchained", "Quentin Tarantino", "Western", 2012, 165),
         new Movie("Tenet", "Christopher Nolan", "Sci-Fi", 2020, 150)
      );
      // Calcola la durata media dei film per regista
      Map<String, Double> averageDurationByDirector =
         movies.stream()
             .collect(Collectors.groupingBy(
                Movie::getDirector,
                Collectors.averagingInt(Movie::getDuration)
             ));
      // Stampa il risultato
      averageDurationByDirector.forEach((director, avgDuration) ->
         System.out.println(director + " -> " + avgDuration + " minuti")
      );
   }
}
```

Esercizio - Libri

- Dato il seguente codice:

```
import java.util.*;
import java.util.stream.*;
public class Main {
  public static void main(String[] args) {
    List<Book> books = Arrays.asList(
      new Book("The Hobbit", "J.R.R. Tolkien", "Fantasy", 12.5, 310, 1937),
      new Book("The Lord of the Rings", "J.R.R. Tolkien", "Fantasy", 25.0, 1178, 1954),
      new Book("1984", "George Orwell", "Sci-Fi", 10.0, 328, 1949),
      new Book("Animal Farm", "George Orwell", "Satire", 7.5, 112, 1945),
      new Book("Dracula", "Bram Stoker", "Horror", 8.5, 418, 1897),
      new Book("The Shining", "Stephen King", "Horror", 15.0, 447, 1977),
      new Book("It", "Stephen King", "Horror", 20.0, 1138, 1986),
      new Book("Dune", "Frank Herbert", "Sci-Fi", 18.0, 412, 1965),
      new Book("The Martian", "Andy Weir", "Sci-Fi", 14.0, 369, 2011),
      new Book("Project Hail Mary", "Andy Weir", "Sci-Fi", 22.0, 496, 2021)
    );
    // TODO (A) Raggruppa per genere e, per ogni genere, stampa il numero di libri
    // TODO (B) Considerando solo i libri con prezzo >= 15,
    //   crea una Map<Autore, List<Titolo>> con i titoli per autore
    // TODO (C) Trova l'autore che ha scritto il maggior numero di pagine totali (somma pagine di tutti i suoi libri)
    //   e stampa "Autore -> totalePagine".
  }
}
```

Esercizio - Libri

```
class Book {
   private String title;
   private String author;
   private String genre;
   private double price;
   private int pages;
   private int year;
   public Book(String title, String author, String genre, double price, int pages, int year) {
      this.title = title;
      this.author = author;
      this.genre = genre;
      this.price = price;
      this.pages = pages;
      this.year = year;  }
   public String getTitle() { return title; }
   public String getAuthor() { return author; }
   public String getGenre() { return genre; }
   public double getPrice() { return price; }
   public int getPages() { return pages; }
   public int getYear() { return year; }
   @Override
   public String toString() {
      return String.format("%s by %s - %s - €%.2f - %d pages - %d",
            title, author, genre, price, pages, year);  }
}
```

Esercizio - Libri

(A) Raggruppa per genere e, per ogni genere, stampa il numero di libri

(B) Considerando solo i libri con prezzo >= 15,

crea una Map<Autore, List<Titolo>> con i titoli per autore

(C) Trova l'autore che ha scritto il maggior numero di pagine totali (somma pagine di tutti i suoi libri) e stampa "Autore -> totalePagine".

Esercizio – Libri - Soluzione

```
import java.util.*;
import java.util.stream.*;
public class Main {
   public static void main(String[] args) {
      List<Book> books = Arrays.asList(…
      // (A) Numero di libri per genere
      Map<String, Long> countByGenre =
         books.stream()
            .collect(Collectors.groupingBy(
               Book::getGenre,
               Collectors.counting()
            ));
      System.out.println("=== (A) Numero di libri per genere ===");
      countByGenre.forEach((genre, count) ->
         System.out.println(genre + " -> " + count)
      );
```

Esercizio – Libri - Soluzione

```
        // (B) Titoli (prezzo >= 15) per autore
        Map<String, List<String>> expensiveTitlesByAuthor =
            books.stream()
                .filter(b -> b.getPrice() >= 15.0)
                .collect(Collectors.groupingBy(
                    Book::getAuthor,
                    Collectors.mapping(Book::getTitle, Collectors.toList())
                ));
        System.out.println("\n=== (B) Titoli (prezzo >= 15) per autore ===");
        expensiveTitlesByAuthor.forEach((author, titles) ->
            System.out.println(author + " -> " + titles)
        );
        // (C) Autore con più pagine totali
        Map<String, Integer> totalPagesByAuthor =
            books.stream()
                .collect(Collectors.groupingBy(
                    Book::getAuthor,
                    Collectors.summingInt(Book::getPages)
                ));
        Map.Entry<String, Integer> topAuthor =
            totalPagesByAuthor.entrySet().stream()
                        .max(Map.Entry.comparingByValue())
                        .orElseThrow();
        System.out.println("\n=== (C) Autore con più pagine totali ===");
        System.out.println(topAuthor.getKey() + " -> " + topAuthor.getValue());
    }
}
```